

Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра интеллектуальных информационных технологий

Лабораторная работа № 4

По дисциплине: «Современные платформы программирования»

За 6 семестр

Выполнил:

Студент 3 курса

Группы ПО-12

Матюх И.М.

Проверил:

Кулик А.Д.

Цель: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

Вариант-2

Анализ активности разработчиков в open-source проекте

Условие:

Напишите Python-скрипт, который анализирует активность разработчиков в указанном публичном GitHub-репозитории и составляет рейтинг самых активных контрибьюторов.

1. Запрашивает у пользователя имя репозитория (например, pallets/flask или microsoft/vscode).
 2. Использует GitHub API для получения списка всех контрибьюторов.
 3. Для каждого контрибьютора собирает данные:
 - ☐ Количество коммитов
 - ☐ Количество открытых pull requests
 - ☐ Количество закрытых pull requests
 - ☐ Количество открытых issues
 - ☐ Количество закрытых issues
 - ☐ Дата последней активности
 4. Определяет самых активных разработчиков по суммарному вкладу.
 5. Строит график вклада разработчиков (matplotlib / seaborn).
- Введите репозиторий для анализа (owner/repo): fastapi/fastapi
Анализируем вклад контрибьюторов в "fastapi/fastapi"...
- ТОП-5 самых активных разработчиков:
1. user1 - 150 коммитов, 40 PR, 20 issues
 2. user2 - 120 коммитов, 35 PR, 15 issues
 3. user3 - 110 коммитов, 25 PR, 10 issues
 4. user4 - 90 коммитов, 20 PR, 8 issues
 5. user5 - 80 коммитов, 15 PR, 5 issues
- Графики активности сохранены в "fastapi_contributors.png"

Код программы

```
import requests
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime
import os
import time

GITHUB_API = "https://api.github.com"

def get_headers():
    headers = {"Accept": "application/vnd.github.v3+json"}
    token = os.getenv("GITHUB_TOKEN") or os.getenv("GH_TOKEN") or
    "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"
    if token:
        headers["Authorization"] = f"token {token}"
```

```

return headers

def handle_rate_limit(response):
    if response.status_code == 403 and "rate limit exceeded" in response.text.lower():
        reset_time = int(response.headers.get("X-RateLimit-Reset", 0))
        if reset_time:
            wait_time = max(reset_time - time.time(), 0) + 1
            print(f"Лимит исчерпан. Ожидание {int(wait_time)} сек...")
            time.sleep(wait_time)
            return True
    return False

def make_request(url, params=None, max_retries=3):
    for attempt in range(max_retries):
        response = requests.get(url, headers=get_headers(), params=params)
        if response.status_code == 200:
            return response
        if handle_rate_limit(response):
            continue
        if response.status_code != 200:
            response.raise_for_status()
    raise Exception("Превышено количество попыток")

def get_contributors(owner, repo):
    url = f"{GITHUB_API}/repos/{owner}/{repo}/contributors"
    params = {"per_page": 100, "sort": "contributions", "order": "desc"}
    response = make_request(url, params)
    return response.json()

def get_user_activity(username):
    user_data = {"commits": 0, "open_prs": 0, "closed_prs": 0, "open_issues": 0,
"closed_issues": 0, "last_activity": None}

    events_url = f"{GITHUB_API}/users/{username}/events"
    response = make_request(events_url, {"per_page": 100})
    if response.status_code == 200:
        events = response.json()
        if events:
            user_data["last_activity"] = events[0].get("created_at", "N/A")

    search_url = f"{GITHUB_API}/search/issues"

    params = {"q": f"author:{username} is:pr is:open", "per_page": 1}
    response = make_request(search_url, params)
    if response.status_code == 200:
        user_data["open_prs"] = response.json().get("total_count", 0)

    params = {"q": f"author:{username} is:pr state:closed", "per_page": 1}
    response = make_request(search_url, params)
    if response.status_code == 200:
        user_data["closed_prs"] = response.json().get("total_count", 0)

    params = {"q": f"author:{username} is:issue is:open", "per_page": 1}
    response = make_request(search_url, params)
    if response.status_code == 200:
        user_data["open_issues"] = response.json().get("total_count", 0)

    params = {"q": f"author:{username} is:issue state:closed", "per_page": 1}
    response = make_request(search_url, params)
    if response.status_code == 200:
        user_data["closed_issues"] = response.json().get("total_count", 0)

    return user_data

def analyze_repo(repo_input):
    if "/" not in repo_input:
        print("Ошибка: Введите репозиторий в формате owner/repo")
        return

```

```

owner, repo = repo_input.split("/")
print(f"\nАнализируем вклад контрибьюторов в \"{repo_input}\"...\")

try:
    contributors = get_contributors(owner, repo)
except Exception as e:
    print(f"Ошибка при получении контрибьюторов: {e}")
    return

top_contributors = []
for i, contributor in enumerate(contributors[:10]):
    username = contributor["login"]
    contributions = contributor.get("contributions", 0)

    try:
        activity = get_user_activity(username)
    except:
        activity = {"commits": contributions, "open_prs": 0, "closed_prs": 0, "open_issues":
0, "closed_issues": 0, "last_activity": None}

    total_score = contributions + activity["open_prs"] + activity["closed_prs"] +
activity["open_issues"] + activity["closed_issues"]

    top_contributors.append({
        "username": username,
        "contributions": contributions,
        "open_prs": activity["open_prs"],
        "closed_prs": activity["closed_prs"],
        "open_issues": activity["open_issues"],
        "closed_issues": activity["closed_issues"],
        "last_activity": activity["last_activity"],
        "total_score": total_score
    })

    print(f"Обработан: {username} ({i+1}/10)")
    time.sleep(1)

top_contributors.sort(key=lambda x: x["total_score"], reverse=True)

print("\nТОП-5 самых активных разработчиков:")
for i, c in enumerate(top_contributors[:5], 1):
    print(f"{i}. {c['username']} - {c['contributions']} коммитов, {c['open_prs']} +
c['closed_prs']} PR, {c['open_issues']} + c['closed_issues']} issues")

create_chart(top_contributors, repo_input)

return top_contributors

def create_chart(contributors, repo_name):
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle(f"Активность контрибьюторов: {repo_name}", fontsize=16, fontweight="bold")

    top5 = contributors[:5]
    names = [c["username"][:12] for c in top5]

    sns.set_style("whitegrid")

    commits = [c["contributions"] for c in top5]
    axes[0, 0].barh(names, commits, color="#2ecc71")
    axes[0, 0].set_xlabel("Количество коммитов")
    axes[0, 0].set_title("Коммиты")
    for i, v in enumerate(commits):
        axes[0, 0].text(v + 1, i, str(v), va="center")

    prs = [c["open_prs"] + c["closed_prs"] for c in top5]
    axes[0, 1].barh(names, prs, color="#3498db")
    axes[0, 1].set_xlabel("Количество PR")
    axes[0, 1].set_title("Pull Requests (открытые + закрытые)")
    for i, v in enumerate(prs):

```

```

        axes[0, 1].text(v + 1, i, str(v), va="center")

issues = [c["open_issues"] + c["closed_issues"] for c in top5]
axes[1, 0].barh(names, issues, color="#e74c3c")
axes[1, 0].set_xlabel("Количество issues")
axes[1, 0].set_title("Issues (открытые + закрытые)")
for i, v in enumerate(issues):
    axes[1, 0].text(v + 1, i, str(v), va="center")

total_scores = [c["total_score"] for c in top5]
axes[1, 1].barh(names, total_scores, color="#9b59b6")
axes[1, 1].set_xlabel("Общий счет активности")
axes[1, 1].set_title("Общий вклад (коммиты + PR + issues)")
for i, v in enumerate(total_scores):
    axes[1, 1].text(v + 1, i, str(v), va="center")

plt.tight_layout()

filename = f"{repo_name.replace('/', '_')}_contributors.png"
plt.savefig(filename, dpi=150, bbox_inches="tight")
print(f"\nГрафики активности сохранены в \"{filename}\"")
plt.close()

if __name__ == "__main__":
    import sys
    if len(sys.argv) > 1:
        repo_input = sys.argv[1]
    else:
        repo_input = input("Введите репозиторий для анализа (owner/repo): ").strip()
    analyze_repo(repo_input)

```

Результат выполнения программы:

```

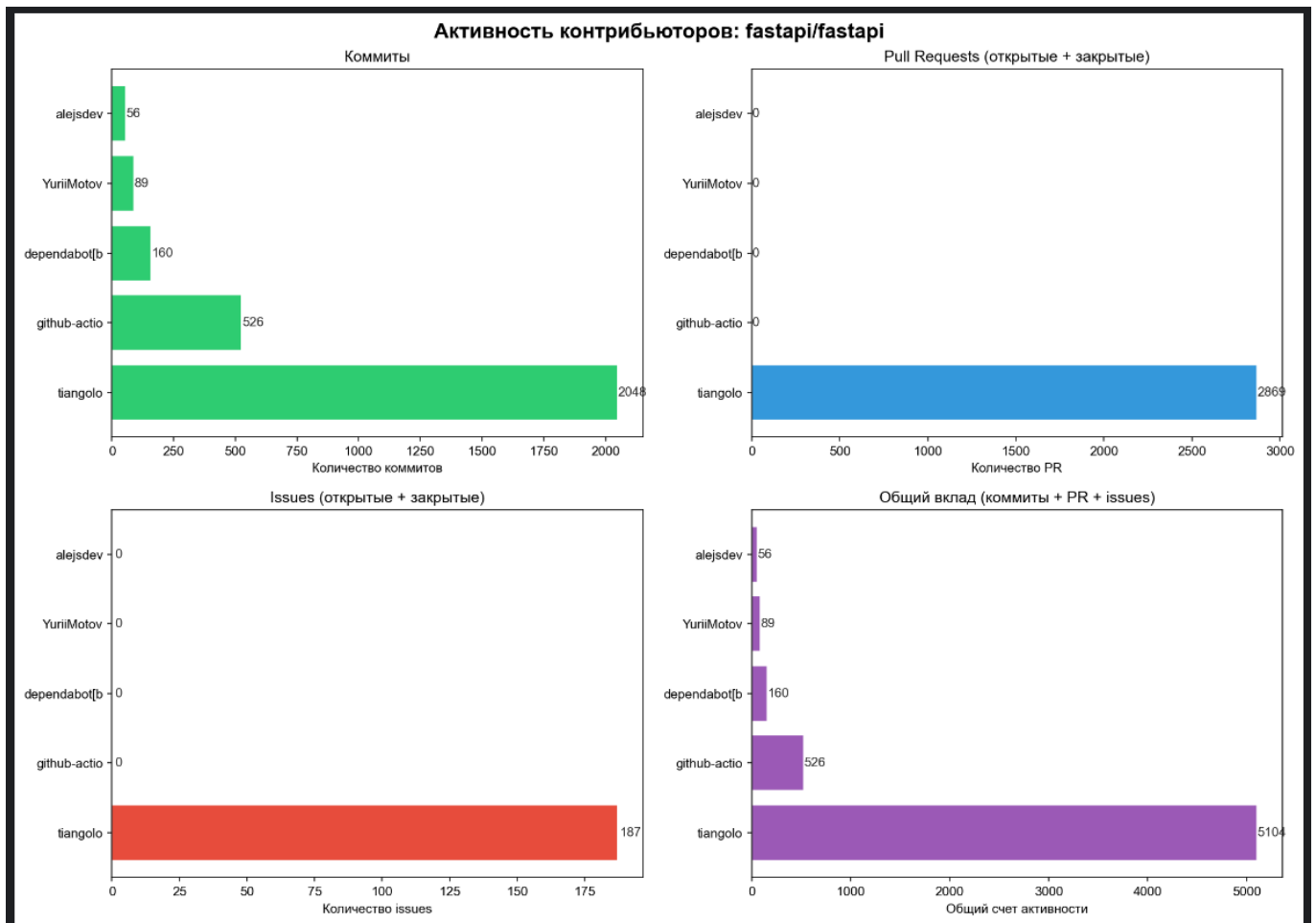
Анализируем вклад контрибьюторов в "fastapi/fastapi"...
Обработан: tiangolo (1/10)
Обработан: github-actions[bot] (2/10)
Обработан: dependabot[bot] (3/10)
Обработан: YuriiMotov (4/10)
Обработан: alejsdev (5/10)
Обработан: pre-commit-ci[bot] (6/10)
Обработан: jaystone776 (7/10)
Обработан: valentinDruzhinin (8/10)
Обработан: seb10n (9/10)
Обработан: Kludex (10/10)

```

ТОП-5 самых активных разработчиков:

1. tiangolo - 2048 коммитов, 2869 PR, 187 issues
2. github-actions[bot] - 526 коммитов, 0 PR, 0 issues
3. dependabot[bot] - 160 коммитов, 0 PR, 0 issues
4. YuriiMotov - 89 коммитов, 0 PR, 0 issues
5. alejsdev - 56 коммитов, 0 PR, 0 issues

Графики активности сохранены в "fastapi_fastapi_contributors.png"



Вывод: научились работать с Github API, приобрели практические навыки написания программ для работы с REST API или GraphQL API